SmartVision-Ed

Computer Vision-Based Student Engagement, Attention Analytics and Multi-Modal Academic Dropout Early Warning System

Project Report

Course: Computer Vision (CSE3002 / CSE4001)

Submitted by

Khushal Mahajan

Registration Number: 24MIM10014

Integrated M.Tech in Computer Science and Engineering

Vellore Institute of Technology (VIT), Bhopal

1. About Me

My name is Khushal Mahajan. I am a Computer Science and Engineering student at VIT, studying in the Integrated M.Tech program, and my registration number is 24MIM10014. My main interests are AI and ML, and I also work as a full-stack developer using the MERN stack (MongoDB, Express, React and Node.js). I am preparing for placements in 2029, so I try to pick projects that let me practise both sides of engineering: building a proper backend and web interface, and applying real algorithms on top of it.

I chose this project for the Computer Vision course because I wanted to do something that was not just a face detector or a toy demo. In every college, teachers and advisors find out that a student is in trouble very late, usually when the attendance notice or the failed midterm result comes. I wanted to see if computer vision, combined with the data a college already has, could give an earlier signal. That idea became SmartVision-Ed.

Through this project I got to use many things in one place: OpenCV and geometry for the vision part, a REST API with FastAPI, a plain JavaScript dashboard, automated testing, and automatic report generation. Most of my earlier work was on the web side, so working with landmarks, camera geometry and signal smoothing was a good learning step for me.

2. Abstract

SmartVision-Ed is a system that watches how attentive a student is during a lecture, using a camera feed, and combines that with normal academic data to estimate how likely the student is to struggle or drop out. For the vision part, the system finds facial landmarks and calculates three things: Eye Aspect Ratio (EAR) to detect drowsiness, Mouth Aspect Ratio (MAR) to detect yawning, and 3D head pose (yaw, pitch, roll) using OpenCV's solvePnP to find out whether the student is looking at the screen or somewhere else. These readings are smoothed using an Exponential Moving Average so that small random changes do not cause false alarms.

The attention result is then fused with lecture attendance, internal assessment marks and LMS activity into one number called the Composite Academic Vulnerability Score (CAVS). Students with a high CAVS, or with attendance below 75 percent, are flagged for early counselling. Privacy was a main design rule: faces are blurred and every student is replaced by a salted SHA-256 pseudonym, so no facial biometric is stored. The project also includes a FastAPI backend, a web dashboard with a What-If simulator, 8 unit tests that all pass, and a script that generates a 15-section PDF report.

3. Problem Statement

In big classrooms, and even more in online or hybrid classes, an instructor cannot watch every student. Whether a student is actually following the lecture is mostly invisible.

The systems that colleges use today to catch struggling students depend on lagging indicators. The attendance rule at university level is 75 percent. When a student falls below it, the debarment notice comes at the end of the semester. Midterm failure reports come after the exam is already over. By then, it is usually too late to fix the situation.

So the problem I tried to solve is this: how can we detect disengagement while it is happening, in a way that respects student privacy, and connect it with academic records so that an advisor gets a clear warning with enough time to act?

4. Objectives

- Detect drowsiness and micro-sleep from eye closure, separating it from normal blinking.
- Detect yawning as a fatigue signal.
- Estimate head direction in 3D and flag when a student is looking away for long periods.
- Classify the attention state as Focused, Distracted, Drowsy, Confused or Neutral.
- Smooth the signals over time so that single noisy frames do not trigger alerts.
- Protect privacy by never storing a face or a real identity.
- Combine vision data with attendance, internal marks and LMS activity into one risk score.
- Raise automatic alerts for debarment risk and dropout risk.
- Give advisors a dashboard where they can test recovery plans (What-If simulator).
- Produce a complete academic report automatically.

5. Scope and Limits

The project covers the full pipeline from a video frame to a final risk alert, along with the dashboard and testing. It is a prototype built for a course. The academic records used are mock records stored in a JSON file (mock_academic_records.json), not real student data from the university. The system should be treated as an advisory tool. It helps a human advisor look in the right direction; it is not meant to take decisions about a student by itself. More on this limitation is given in Section 12.

6. Background

A few ideas from existing work are used in this project.

Eye Aspect Ratio. Soukupova and Cech (2016) showed that the ratio between the vertical and horizontal distances of the eye landmarks drops quickly when the eye closes, which makes it a simple and reliable blink and drowsiness measure. I use their formula directly.

Mouth Aspect Ratio. MAR is the same idea applied to the mouth. When the mouth opens wide for a long moment, the ratio rises sharply, which is what happens during a yawn.

Head pose with PnP. If we know where a few facial points are in a generic 3D face model and where they appear in the 2D image, the Perspective-n-Point problem gives the rotation and translation of the head relative to the camera. OpenCV provides this as cv2.solvePnP. The rotation can then be converted into yaw, pitch and roll angles.

Exponential Moving Average. A simple filter that gives more weight to recent values while still remembering the past. It is cheap to compute and works well for real-time streams.

Pseudonymization. Instead of storing a name or face, a salted hash such as SHA-256 turns an identity into a token that cannot be reversed easily but stays the same for the same person.

7. System Architecture

The system is divided into four layers, so each part can be tested and changed on its own. Data moves from the top layer to the bottom one.

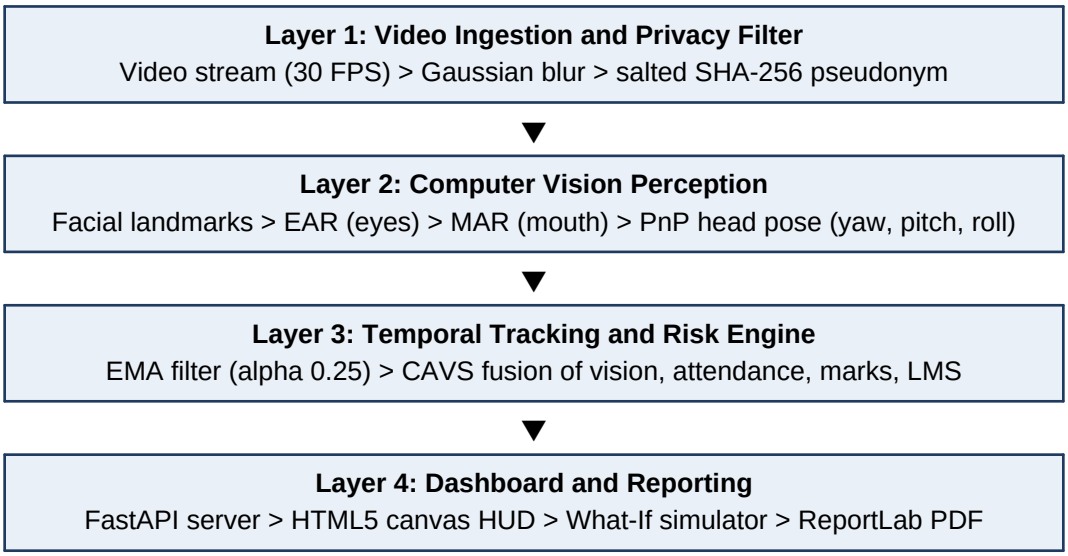

Figure 1: Four-layer architecture of SmartVision-Ed

Layer 1 takes the video stream at about 30 FPS, blurs the face region with a Gaussian filter and gives each student a pseudonym like ANON-8B4E19C0. **Layer 2** uses facial landmarks to calculate EAR, MAR and the head pose angles. **Layer 3** smooths the readings with the EMA filter (alpha = 0.25) and calculates the CAVS by combining the vision score with attendance, internal marks and LMS activity. **Layer 4** is a FastAPI server with a browser dashboard showing a live HUD on an HTML5 canvas and the What-If simulator, plus the ReportLab code that builds the PDF.

8. Methodology

8.1 Eye Aspect Ratio (EAR)

For the six landmarks p1 to p6 around one eye:

```
EAR = ( |p2 - p6| + |p3 - p5| ) / ( 2 x |p1 - p4| )
```

The top part is the two vertical distances and the bottom part is the horizontal width of the eye. An open eye gives roughly 0.28 to 0.32. A blink makes the value drop below about 0.22 for a few frames (six or fewer). Real drowsiness is different: the value stays below 0.18 for more than 14 frames in a row. This frame count is what separates a sleepy student from someone who just blinked.

8.2 Mouth Aspect Ratio (MAR)

```
MAR = ( |m2 - m8| + |m3 - m7| + |m4 - m6| ) / ( 2 x |m1 - m5| )
```

Normal mouth position stays below 0.40. When MAR goes above 0.65, it is counted as a yawn event.

8.3 Head Pose with solvePnP

A fixed set of 3D model points of a generic face (nose tip, chin, eye corners, mouth corners) is matched with the same points detected in the image. cv2.solvePnP returns a rotation vector, which is converted to a rotation matrix and then decomposed into Euler angles: yaw (left-right), pitch (up-down) and roll (tilt). If a student keeps their head turned far from the screen for a long time, it is treated as distraction. In the unit test, a student looking straight at the camera should have absolute yaw and pitch values below 20 degrees.

8.4 Temporal Smoothing

Raw values jump around from frame to frame because of lighting, small movements and detection noise. Each signal is smoothed with:

```
S(t) = alpha x X(t) + (1 - alpha) x S(t-1), alpha = 0.25
```

A lower alpha means a smoother but slower reaction. I used 0.25 as a balance. The tracker also keeps a running count of fatigue events during a session.

8.5 Attention State Classifier

The classifier uses simple rules on several cues together, and it picks one of five states:

- **Focused:** eyes open, head facing the screen, no yawn.
- **Distracted:** head turned away for long.
- **Drowsy:** EAR below 0.18 for more than 14 frames.
- **Confused:** the in-between case where the head and expression suggest the student is struggling.
- **Neutral:** nothing strong in either direction.

I kept it rule-based on purpose. The rules can be explained to anyone, which matters when the output is going to be used by an advisor.

8.6 Privacy Filter

Two things happen before anything is shown or stored. First, the face region is blurred with a Gaussian filter. Second, the student's identity is converted to a pseudonym using SHA-256 with a secret salt. The same student always gets the same token, so trends can be tracked over time, but the token does not reveal who the person is. No facial image or biometric template is saved anywhere in the system.

8.7 Composite Academic Vulnerability Score (CAVS)

The four inputs are each on a 0 to 100 scale where higher is better:

- Vision attention score (Vatt), weight 0.30
- Lecture attendance (Att), weight 0.25
- Internal assessment marks (Imk), weight 0.25
- LMS activity (Lms), weight 0.20

```
CAVS = 100 - ( 0.30 x Vatt + 0.25 x Att + 0.25 x Imk + 0.20 x Lms )
```

So a high CAVS means high vulnerability. The result is read like this:

CAVS range	Level	Meaning
Below 30	Low risk	Good engagement and steady academic standing
30 to below 58	Moderate risk	Early signs of disengagement; proctor should keep an eye
58 or more, or attendance below 75	Critical	High chance of dropout or debarment; counselling needed at once

The attendance rule works separately from the score. Even if the CAVS looks fine, a student below 75 percent attendance is still flagged for debarment risk. The weights come from my own judgment: the visual signal is the newest information so it gets the biggest share, and the other three are balanced against each other. They are not learned from real data, and they can be changed in risk_analyzer.py.

9. Implementation

9.1 Project Structure

Folder / file	Purpose
core/vision_engine.py	EAR, MAR, PnP head pose and the affect classifier
core/attention_tracker.py	EMA smoothing and session-level totals
core/risk_analyzer.py	CAVS calculation and alerts
core/privacy_filter.py	Blurring and salted SHA-256 pseudonyms
models/data_schemas.py	Pydantic schemas for telemetry, pose, eye and risk profile
api/app.py	FastAPI endpoints and static file serving
web/	index.html, styles.css, dashboard.js for the dashboard
data/	Mock academic records and generated charts
tests/	Unit tests for vision and risk modules
scripts/	generate_charts.py and build_report_pdf.py
reports/	The generated PDF report

9.2 Technologies Used

Area	Tools
Language	Python 3.13
Computer vision	OpenCV (opencv-python-headless), NumPy
Math and ML	Scikit-Learn, SciPy
Backend	FastAPI, Uvicorn, Pydantic
Frontend	HTML5, CSS3 (dark mode, glassmorphism), Canvas API, JavaScript
Plots	Matplotlib, Pillow
Reports	ReportLab 5.0.1
Testing	Python unittest

Since I already work with web development, I kept the frontend in plain HTML, CSS and JavaScript with no framework, so that the dashboard can also be opened directly from web/index.html without running the server.

9.3 Backend

The FastAPI app exposes the endpoints that the dashboard calls, and Pydantic schemas make sure that every telemetry packet and risk profile has the right fields and types. The same app also serves the web

folder, so one command is enough to start everything.

9.4 Dashboard

The dashboard shows a live canvas HUD with the readings (EAR, MAR, head angles, attention state) and charts of the risk. The What-If simulator has sliders for attendance and marks. An advisor can move a slider and watch the CAVS change immediately, which helps to answer the question of what the student needs to reach to get out of the critical zone.

9.5 Report Generation

scripts/build_report_pdf.py uses ReportLab to build a 15-section PDF with the diagrams, tables and test results. It uses a numbered canvas so that every page has a proper page number. The figures it uses are produced by scripts/generate_charts.py: the architecture diagram, the workflow diagram, the EAR drowsiness curve and the CAVS distribution.

10. How to Run

1. Clone the repository (github.com/khushal-mahajan/SmartVision-Ed) and go into the folder.
2. Create and activate a virtual environment (venv).
3. Install the packages with: `python -m pip install -r requirements.txt`
4. Start the server with: `python -m uvicorn api.app:app --host 127.0.0.1 --port 8000 --reload`
5. Open http://127.0.0.1:8000 in a browser. The dashboard can also be opened directly from web/index.html.
6. To regenerate figures and the PDF: run scripts/generate_charts.py and then scripts/build_report_pdf.py.

11. Testing and Results

The tests are run with: `python -m unittest discover tests -v`

Test	What it checks	Result
test_ear_calculation_open_and_closed_eye	Open eye gives EAR above 0.25 and closed eye gives below 0.15	Passed
test_mar_calculation_normal_and_yawn	Normal mouth below 0.40 and yawn above 0.65	Passed
test_pnp_head_pose_estimation	Yaw and pitch stay under 20 degrees for a frontal face	Passed
test_privacy_pseudonymization	Same input gives the same salted SHA-256 token	Passed
test_temporal_attention_tracker	EMA smoothing and fatigue accumulation work correctly	Passed
test_cavs_calculation	CAVS behaves correctly at boundary values	Passed
test_evaluate_student_critical_attendance_debarment	Attendance below 75 sets the debarment flag	Passed
test_evaluate_student_high_achiever	A strong student is evaluated as low risk	Passed

All 8 of 8 tests pass. These tests check that the math and the logic are correct. They do not measure how accurate the system would be in a real classroom with real students. I did not run a large field study, so I am not claiming any accuracy percentage for the detection of drowsiness or the prediction of dropout.

What the project does show is that the full pipeline works from end to end: the ratios are computed correctly, the thresholds separate normal and abnormal cases in the test conditions, the privacy step gives stable pseudonyms, and the risk engine gives the expected level for the example students, including the automatic debarment flag.

12. Limitations

- **Mock data.** The academic records are made-up samples. The risk weights have not been checked against real dropout outcomes.
- **Fixed thresholds.** The values 0.18 for EAR and 0.65 for MAR work in typical conditions but can be different for different people, for example for people with naturally narrow eyes, or for people wearing glasses.
- **Camera and light.** Poor lighting, low resolution, masks, or a camera at a bad angle will reduce the quality of landmark detection.
- **Looking away is not always distraction.** A student may be taking notes or thinking. A rule-based classifier cannot know the reason.
- **Rules instead of learning.** The Confused state in particular is a rough guess from simple cues.
- **Ethics.** Monitoring students by camera is sensitive. Even with blurring and pseudonyms, a real deployment needs clear consent, a stated purpose, limits on who can see the data, and a rule that the score is never used as punishment. A human must always make the final decision.

13. Future Work

- Test with real classroom recordings, with consent, and measure the detection accuracy.
- Calibrate the thresholds per student during the first minutes of a session.
- Replace the hand-set CAVS weights with weights learned from real historical data, using the Random Forest model from Scikit-Learn that is already part of the toolset.
- Add gaze estimation, not only head pose.
- Handle multiple students in one frame in a more robust way.
- Run the vision part on the student's device so that video never leaves the room.
- Add login and role-based access for advisors, and connect with a real LMS.

14. Conclusion

SmartVision-Ed shows that a college can use computer vision and its own academic data together to see a problem before it becomes a failure. Simple, well-known tools were enough for this: EAR for drowsiness, MAR for yawning, solvePnP for head direction, an EMA filter for noise, and a weighted score for risk. The privacy layer makes sure the system works without storing any face.

For me, this project connected what I learned in the Computer Vision course with the web development I already do. I built the maths, the API, the dashboard, the tests and the report generator myself, and I understand the weak points of the system as well as its strong points. If it were taken further, it should be with real data, proper consent and always with a human advisor in the loop.

15. References

1 T. Soukupova and J. Cech, Real-Time Eye Blink Detection using Facial Landmarks, 21st Computer Vision Winter Workshop, 2016.

2 G. Bradski, The OpenCV Library, Dr. Dobb's Journal of Software Tools, 2000. Documentation for cv2.solvePnP: docs.opencv.org.

3 V. Lepetit, F. Moreno-Noguer and P. Fua, EPnP: An Accurate O(n) Solution to the PnP Problem, International Journal of Computer Vision, 2009.

4 FastAPI documentation: fastapi.tiangolo.com

5 Pydantic documentation: docs.pydantic.dev

6 ReportLab documentation: reportlab.com

7 Scikit-learn documentation: scikit-learn.org

8 SmartVision-Ed repository and README: github.com/khushal-mahajan/SmartVision-Ed